CHAPTER 1

INTRODUCTION

1.1 BACKGROUND OF THE STUDY

Convolutional neural networks (CNNs) have revolutionized the field of image classification by enabling deep architectures to automatically learn hierarchical features from raw pixel data. Since the breakthrough of AlexNet on the ImageNet challenge in 2012, CNNs have become the default choice for computer vision tasks because of their ability to extract both local and global patterns, and their scalability for high-dimensional data. Transfer learning is a technique where knowledge gained from training large-scale CNNs on extensive datasets (like ImageNet) is harnessed to solve new, related tasks with smaller datasets. Pretrained models such as VGG, ResNet, Inception, MobileNet, and Xception have convolutional layers that can recognize general visual features; these layers are reused, and only some or all of the new task-specific layers are retrained on the new data, leading to faster convergence and better accuracy with less data and computational effort.

Hyperparameter optimization—systematic tuning of parameters such as learning rate, dropout, number of trainable layers, batch size, and optimizer—is critical for maximizing CNN performance, both when training from scratch and in transfer learning scenarios. The process is computationally intensive and often relies on search algorithms such as Grid Search, Random Search, Bayesian optimization, and advanced measures like ASHA to efficiently explore the vast combinations of possible hyperparameters. The contemporary research focus, as exemplified by your base paper, is on systematically understanding and optimizing these hyperparameters, especially in the context of fine-tuned (transfer-learned) networks. Studies show that careful tuning can yield substantial accuracy improvements; moreover, using balanced subsets of training data for optimization and employing modern search strategies leads to improved generalization and resource efficiency. The findings underscore that no universal hyperparameter set fits all contexts, and optimal values are dataset- and task-dependent.

1.2 PROBLEM STATEMENT

Image classification is a fundamental task in computer vision, yet achieving high accuracy while maintaining computational efficiency remains a key challenge. Traditional CNN models trained from scratch require extensive parameter tuning and large amounts of data to generalize effectively, often leading to longer training times and overfitting issues. Conversely, pre-trained deep learning models such as MobileNetV2 leverage transfer learning to accelerate training and improve performance, but their effectiveness on smaller datasets like CIFAR-10 and their adaptability to task-specific domains remain uncertain.

The central problem addressed in this project is to determine the effectiveness of building a CNN from scratch versus employing a pre-trained MobileNetV2 model for CIFAR-10 image classification, while systematically applying hyperparameter tuning and data augmentation to optimize model performance. Specifically, the study aims to evaluate trade-offs in accuracy, computational cost, convergence speed, and generalization ability between these two approaches, thereby identifying the most efficient strategy for achieving high-performing image classification models.

1.3 OBJECTIVE OF THE PROJECT

The primary objective of this project is to investigate and compare the performance of a CNN model built from scratch with a pre-trained MobileNetV2 architecture for image classification on the CIFAR-10 dataset. The study aims to design and train a custom CNN to evaluate its accuracy, convergence behavior, and generalization ability, while also implementing transfer learning through MobileNetV2 with variations in fine-tuning of its base layers.

To optimize both approaches, systematic hyperparameter tuning is carried out using Keras Tuner, focusing on key parameters such as the number of convolutional filters, dense layer units, learning rates, and training strategies. In addition, data augmentation techniques, including rotation, shifting, and flipping, are employed to enhance model robustness and minimize overfitting. Ultimately, the objective is to derive meaningful insights into the relative effectiveness of training CNNs from scratch versus leveraging pre-trained models, while providing practical recommendations for selecting suitable architectures and optimization strategies in real-world image classification tasks.

1.4 SCOPE AND LIMITATIONS

The scope of this project focuses on the application of deep learning models for image classification using the CIFAR-10 dataset, which consists of 60,000 images across 10 object categories. The study is confined to evaluating two specific approaches: a CNN model developed from scratch and a transfer learning approach using the pre-trained MobileNetV2 architecture. The project emphasizes the role of hyperparameter tuning through Keras Tuner, exploring parameters such as filter size, dense layer units, learning rates, and training strategies. Data augmentation techniques including rotation, shifting, and flipping are also applied to improve generalization and minimize overfitting. The comparative analysis covers key aspects such as accuracy, convergence rate, computational efficiency, and stability across training and validation phases. This research is intended to provide insights into the trade-offs between building CNNs from scratch and employing transfer learning models, thereby contributing recommendations for practical applications in image classification tasks.

However, the study also has certain limitations. First, the experiments are limited to the CIFAR-10 dataset, which, while widely used as a benchmark, may not fully represent the complexity of large-scale real-world image datasets. Second, the number of hyperparameter tuning trials and epochs is constrained due to computational resources, meaning that deeper exploration of the hyperparameter space may yield different results. Third, only one pre-trained model, MobileNetV2, is considered, whereas other architectures such as ResNet, EfficientNet, or Inception could provide broader insights. Additionally, the project does not focus on deployment, model compression, or inference optimization, which are important considerations for practical real-world applications. These limitations highlight opportunities for extending the study in future work, including testing on larger datasets, experimenting with a wider range of architectures, and evaluating models in terms of scalability and deployment efficiency.

1.5 SIGNIFICANCE OF THE STUDY

This study holds significant importance in the field of deep learning and computer vision as it addresses the practical challenges of choosing between building a CNN from scratch and adopting transfer learning approaches for image classification. By conducting a comparative analysis on the CIFAR-10 dataset, the project provides valuable insights into how hyperparameter tuning and data augmentation influence the performance, stability, and efficiency of both models. The findings highlight not only the strengths of pre-trained models such as MobileNetV2 in terms of faster convergence and higher baseline accuracy but also the relevance of custom-built CNNs when flexibility and domain-specific design are required.

The significance of this work extends to both academic research and practical applications. For researchers, it offers a framework for systematically evaluating and optimizing deep learning models through hyperparameter tuning, serving as a reference for similar comparative studies. For practitioners, particularly those working in resource-constrained environments, the study demonstrates trade-offs in computational cost, training time, and performance, helping them make informed decisions when selecting model architectures for real-world image classification tasks. Furthermore, this study emphasizes the importance of leveraging pre-trained models to accelerate training and improve generalization, while also showcasing how carefully designed CNNs from scratch can still remain competitive. Ultimately, the project contributes to advancing knowledge on the effective use of deep learning models, bridging the gap between theoretical exploration and practical deployment.

CHAPTER 2

LITERATURE REVIEW

2.1 REVIEW AND GAP IN EXISTING RESEARCH

Wojciuk et al. (2024) — Improving classification accuracy of fine-tuned CNN models: Impact of hyperparameter optimization. This paper systematically compares Grid Search, Random Search, Bayesian Optimization and ASHA for hyperparameter tuning of fine-tuned CNNs over several datasets (CIFAR-100, Stanford Dogs, MIO-TCD). It highlights the importance of learning rate and input size (via fANOVA), shows ASHA/Bayesian often outperform grid/random under limited budget, and demonstrates that a balanced subset can be used to find good hyperparameters. This is your base methodology and motivates using ASHA/Bayesian + fANOVA-style analysis in your project. [1]

Krizhevsky (2009) — CIFAR datasets / Tiny images (CIFAR-10/100). The CIFAR datasets are small-color natural image benchmarks (32×32) widely used to evaluate CNN architectures. They provide a standardized testbed for comparing custom CNNs and transfer learning approaches (CIFAR-10 is your experimental dataset), and highlight challenges around small image size, overfitting, and the need for augmentation. [2]

Snoek et al. (2015) — Scalable Bayesian optimization for hyperparameter tuning. Introduces practical Bayesian optimization techniques for expensive black-box functions (e.g., model validation accuracy) and shows BO often needs fewer evaluations than random/grid search. Directly relevant for implementing Bayesian hyperparameter search in Keras Tuner and for justifying BO as a candidate method in your experiments. [3]

Li et al. (2018) / Jamieson et al. — ASHA / Hyperband family (successive halving). ASHA/Hyperband accelerate tuning by early-stopping poor configurations and allocating more budget to promising trials. Their resource-aware strategy is ideal when GPU/epoch budgets are limited — a key rationale for including ASHA in your comparative study. [4]

Falkner, Klein & Hutter (2018) — BOHB: combining Bayesian Optimization and Hyperband. BOHB blends BO's model-based sampling with Hyperband's adaptive resource allocation, achieving robust tuning across budgets. This hybrid idea supports exploring combinations of methods in your project (e.g., efficient search under limited epochs). [5]

Goodfellow, Bengio & Courville — Deep Learning (2016). The standard text covering CNN fundamentals, optimization techniques, regularization (dropout, batch norm), and best practices for training deep models. Useful for grounding architectural/design choices in the custom CNN and for interpreting hyperparameter impacts. [6]

He et al. (2016) — ResNet: Deep residual learning. Demonstrates residual connections enabling very deep nets; provides insight into architecture choices and why transfer learning from deep backbones often yields strong features. Helps explain why pre-trained deep models (though heavier than MobileNet) generalize well. [7]

Sandler et al. (2018) / MobileNetV2 paper. Introduces MobileNetV2's inverted residuals and linear bottlenecks to create an efficient model for mobile/edge. Relevant because MobileNetV2 is your pre-trained backbone — its design explains why it converges faster and is compute-efficient compared to de-novo CNNs. [8]

Tan & Le (2019) — EfficientNet: Model scaling that matters. Shows principled compound scaling of depth/width/resolution yields state-of-the-art efficiency. Cited as an example of other efficient pre-trained architectures you could explore in future work beyond MobileNetV2. [9]

Hutter, Hoos & Leyton-Brown (2014) — fANOVA: assessing hyperparameter importance. Presents a methodology for quantifying hyperparameter importance via functional ANOVA. Directly used by Wojciuk et al. to reduce search space, and a useful tool for your project to prioritize tuning dimensions (e.g., learning rate, dropout). [10]

Bergstra & Bengio (2012) — Random Search for Hyper-Parameter Optimization. Demonstrates that random search often outperforms grid search because it better explores important hyperparameter dimensions. Motivates including Random Search as a baseline in your comparisons. [11]

Bergstra et al. / Hyperopt (2013) — Practical Bayesian optimization implementations. Discusses early practical libraries and algorithms for automated tuning; relevant for implementation choices (e.g., Keras Tuner's Bayesian tuner or wrappers around Hyperopt). [12]

Keras Tuner / O'Reilly / TensorFlow docs (practical guides). Practical guidance on integrating hyperparameter search into TensorFlow workflows (RandomSearch, Bayesian, Hyperband). Useful for your code orchestration and experiment reproducibility. [13]

Zela, Klein, Falkner & Hutter (2018) — Joint architecture and hyperparameter search. Argues that architecture and hyperparameter search can be combined for better results. For your study this supports the idea that both CNN design choices (filters, layers) and training hyperparameters (lr, dropout) should be tuned jointly when possible. [14]

Garbin et al. / Dropout vs BatchNorm empirical study (2020). Empirically examines how dropout and batch normalization affect training stability and generalization. This informs your decisions about where to include dropout layers in the scratch CNN versus relying on batchnorm or other regularizers. [15]

Kingma & Ba (2015) — Adam optimizer. Widely used adaptive optimizer with strong empirical performance; frequently a default for CNNs and used in many tuning studies (including Wojciuk et al.); explains why Adam often appears in optimized configurations. [16]

Srivastava et al. (2014) — Dropout: a simple way to prevent neural networks from overfitting. Foundational paper on dropout regularization and its impact on generalization — directly relevant to your tuning of dropout rates in both models. [17]

Shorten & Khoshgoftaar (2019) — A survey on Image Data Augmentation for Deep Learning. Comprehensive survey of augmentation strategies and their empirical benefits. Provides theoretical and empirical support for using rotation, shifting, flipping in your pipelines and for exploring augmentation in tuning. [18]

Perez & Wang (2017) — The Effectiveness of Data Augmentation in Image Classification using Deep Learning. Demonstrates quantitative improvements in generalization from common augmentation techniques, supporting their inclusion as part of model training and as a hyperparameter dimension. [19]

Zhang et al. (2017) — mixup: Beyond Empirical Risk Minimization. Introduces mixup augmentation which blends pairs of images and labels; often improves robustness. Suggested as a future augmentation strategy to explore beyond geometric transforms. [20]

Howard et al. (2017) — MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. Earlier MobileNet work emphasizing depthwise separable convolutions to reduce parameters and compute. Reinforces why MobileNet variants are good transfer-learning candidates in resource-constrained experiments. [21]

Snoek, Larochelle & Adams (2012) — Practical Bayesian optimization tutorial. Tutorial-style exposition of BO for expensive functions (like training CNNs), useful for understanding acquisition functions and exploration/exploitation tradeoffs in model search. [22]

Li et al. (2020) — On the importance of dataset composition for hyperparameter tuning (subset strategies). Studies show that hyperparameter optimization on a carefully chosen subset (balanced or reweighted) can produce configurations that generalize to the full set while saving compute—matching Wojciuk et al.'s findings and motivating your experiments with subsets. [23]

Krizhevsky, Sutskever & Hinton (2012) — ImageNet classification with deep convolutional neural networks (AlexNet). Landmark that popularized deep CNNs, introduced practices (ReLU, dropout, data augmentation) that underpin modern training and tuning — historical context for why transfer learning from ImageNet can be so powerful. [24]

Loshchilov & Hutter (2016) — SGDR: Stochastic Gradient Descent with Warm Restarts (learning-rate schedules). Proposes cyclical/annealing learning-rate schedules that often yield better minima; indicates that learning-rate schedule is an influential hyperparameter family to consider in tuning. [25]

Molchanov et al. / Han et al. — Model compression & pruning studies. Research shows compressed/pruned models can retain accuracy with smaller compute; relevant to comparing inference cost and deployment feasibility between your custom CNN and MobileNetV2 (as future direction). [26]

Ottoni et al. (2023) — Hyperparameter tuning of CNNs for domain-specific image classification. A domain-applied study showing that careful tuning significantly boosts performance in specialized tasks (construction images), reinforcing the practical value of systematic tuning beyond benchmarks. [27]

LeCun, Bengio & Hinton (2015) — Deep learning (Nature). High-level overview of deep learning progress and core ideas; useful as a conceptual anchor for your report's literature intro. [28]

Li et al. (2018) — Parallelizing Hyperband for large-scale tuning. Discusses parallel hardware-aware strategies for efficient search which informs experiment design when run on multi-GPU or cloud resources. [29]

Recent surveys on AutoML / Neural Architecture Search (NAS) (2018–2022). Summarize automated methods for selecting architectures and hyperparameters together; provide context for why hyperparameter tuning remains a critical, more accessible step for many practitioners who cannot run full NAS. [30]

CHAPTER 3

METHODOLOGY

3.1 RESEARCH DESIGN

3.1.1 Research Goal

The primary goal of this research is to investigate and compare the effectiveness of a custom Convolutional Neural Network (CNN) trained from scratch versus a pre-trained MobileNetV2 model with transfer learning for image classification on the CIFAR-10 dataset. The study aims to:

1. Evaluate the classification performance of both models in terms of accuracy, loss, and generalization on unseen data.

2. Optimize model performance through hyperparameter tuning, including learning rate, number of dense units, and trainable layers in the pre-trained model.

3. Analyze the benefits of transfer learning compared to training a CNN from scratch in terms of training efficiency, convergence speed, and final accuracy.

4. Provide insights into practical implementation strategies for building high-performing deep learning models on small to medium-sized image datasets.

3.1.2 Research Type

- **Applied Experimental Research**: This study applies deep learning techniques to solve an image classification problem using the CIFAR-10 dataset.
- **Comparative Study**: The research compares the performance of a **custom CNN** trained from scratch versus a **pre-trained MobileNetV2** model with transfer learning and hyperparameter tuning.

3.1.3 Research variables

Variable Type	Variable	Description
Independent	Model Type	CNN from scratch, Pre-trained MobileNetV2
Independent	Hyperparameters	Learning rate, number of dense units, trainable base, number of filters
Dependent	Performance Metrics	Accuracy, Loss, Training Time
Control	Dataset & Pre-processing	CIFAR-10, Data normalization, Data augmentation

Table 3.1 Overview of variables

1. Environment Setup

Install and import the necessary libraries: TensorFlow, Keras, Keras Tuner, NumPy, Matplotlib. Ensure reproducibility by fixing random seeds for TensorFlow, NumPy, and Python's random module.

2. Data Preparation

- **Dataset**: CIFAR-10, consisting of 60,000 32×32 color images in 10 classes (50,000 training and 10,000 test images).
- **Normalization**: Scale pixel values to [0,1] for the CNN model. For MobileNetV2, use the preprocess_input function specific to the architecture.
- **One-hot Encoding**: Convert class labels into one-hot vectors to be compatible with categorical cross-entropy loss.

3. Data Augmentation

- Use ImageDataGenerator to apply real-time data augmentation to increase dataset diversity and reduce overfitting.
- Augmentation techniques applied:
    - **Rotation:** Randomly rotate images within 15 degrees.
    - **Width & Height Shift:** Randomly shift images up to 10% of width/height.
    - **Horizontal Flip:** Randomly flip images horizontally.

4. Model Design

A. Custom CNN Model (from scratch)

- Stack of **Conv2D layers** with ReLU activations, followed by **MaxPooling2D** layers for spatial downsampling.
- **Dropout layers** added after convolutional and dense layers to reduce overfitting.
- Final **Dense layer** with softmax activation to predict 10 classes.
- Hyperparameters tuned:
    - Number of filters in first convolutional layer.
    - Number of units in the dense layer.
    - Learning rate of Adam optimizer.

B. Pre-trained MobileNetV2 Model

- Load MobileNetV2 with include_top=False to use as a feature extractor.
- **GlobalAveragePooling2D** applied after the base model.
- Add **Dense layer + Dropout** before the output layer.
- Hyperparameters tuned:
    - Whether to fine-tune the base model (trainable_base).
    - Number of units in the dense layer.
    - Learning rate of Adam optimizer.

5. Hyperparameter Tuning

- Use **Keras Tuner** with RandomSearch:
    - Objective: Maximize **validation accuracy**.
    - Trials: Initially 2 (can increase for better optimization).
- Each model is rebuilt and trained with different hyperparameter combinations.
- Early stopping is applied to avoid overfitting during tuning.

6. Model Training

- Train each model using the **augmented dataset** from ImageDataGenerator.
- Monitor **accuracy** and **loss** for both training and validation sets.
- Use **EarlyStopping** to restore the best weights when validation loss stops improving.

7. Evaluation

- After hyperparameter tuning, select the **best hyperparameters** for the final model.
- Retrain the best model on the full training set.
- Evaluate on the test set to obtain **final test accuracy** and **loss**.

8. Visualization

- Plot **training & validation accuracy** and **loss** for each trial.
- Compare performance between:
    - CNN from scratch.
    - Pre-trained MobileNetV2 with/without fine-tuning.
- Helps analyze the benefits of transfer learning and hyperparameter optimization.

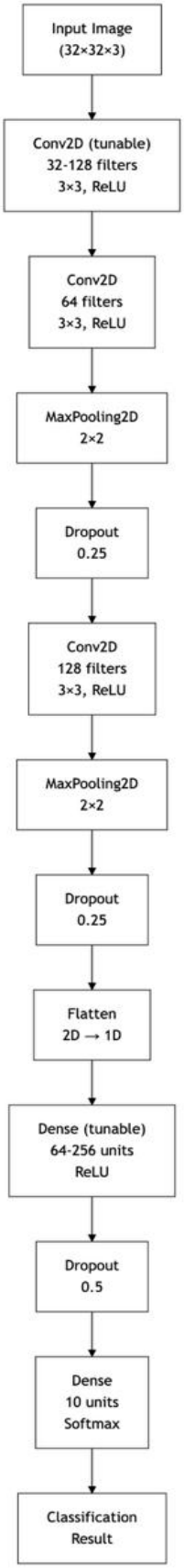

Fig 3.1 CUSTOMIZED CNN ARCHITECTURE DIAGRAM

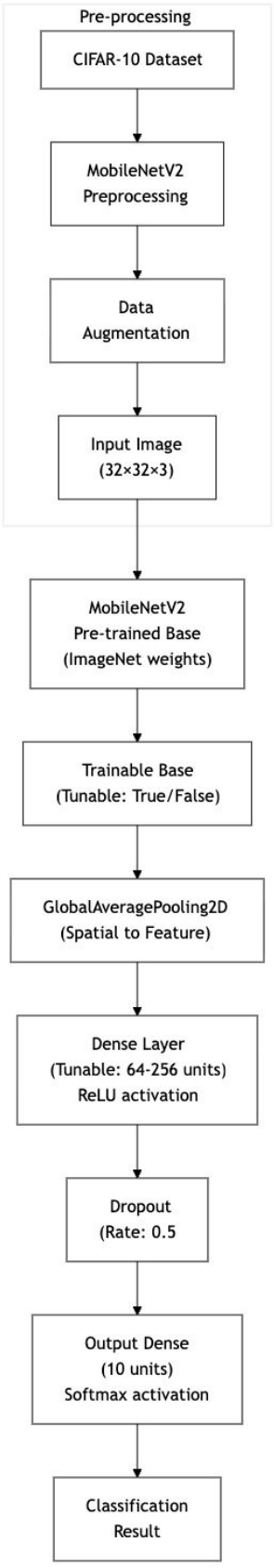

Fig 3.2 PRE-PROCESSING CNN ARCHITECTURE DIAGRAM

CHAPTER 4

DATA ANALYSIS, RESULT AND DISCUSSION

4.1 DATA ANALYSIS

4.1.1 Dataset Overview

The dataset used for this study is CIFAR-10 (Canadian Institute for Advanced Research), a widely recognized benchmark for image classification tasks. It is publicly available through the Keras datasets module and was originally collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. CIFAR-10 contains a total of 60,000 color images of size 32×32 pixels with three color channels (RGB), divided into 50,000 training images and 10,000 test images. The dataset is organized into 10 mutually exclusive classes, each containing exactly 6,000 images to ensure class balance. The classes include airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck, representing a diverse set of real-world objects.

The images in CIFAR-10 present several challenges for classification, such as low resolution, high intra-class variation, and inter-class similarities, for instance, between cats and dogs or automobiles and trucks. To prepare the data for deep learning models, preprocessing steps are applied. For CNNs trained from scratch, pixel values are normalized to the range [0,1], while for pre-trained models like MobileNetV2, the preprocess_input() function is used to match the network's expected input distribution. Labels are converted to one-hot encoding to be compatible with categorical cross-entropy loss. Additionally, data augmentation is performed using rotation, width and height shifts, and horizontal flips to enhance model generalization and reduce overfitting.

CIFAR-10 is significant for this study because it provides a standardized, balanced, and diverse set of natural images, making it suitable for evaluating and comparing the performance of both custom CNN models and transfer learning approaches. Its controlled yet challenging nature allows researchers to assess model accuracy, learning efficiency, and robustness in image classification tasks.

Dataset Description

- **Type**: Image classification dataset
- **CIFAR-10** (Canadian Institute for Advanced Research)
- **Size**: 60,000 color images
    - **Training set**: 50,000 images
    - **Test set**: 10,000 images
- **Image dimensions**: 32 × 32 pixels
- **Channels**: 3 (RGB)
- **Classes**: 10 mutually exclusive categories
- **Format**: Each image is represented as a 32×32×3 array of pixel values.
- **Label Representation**: Integer (0–9), typically converted to one-hot **e**ncoding for training deep learning models.
- The CIFAR-10 dataset is publicly available through the Keras datasets module (tf.keras.datasets.cifar10) and was originally collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.
- URL: https://www.cs.toronto.edu/~kriz/cifar.html

Class Label	Description
0	Airplane
1	Automobile
2	Bird
3	Cat
4	Deer
5	Dog
6	Frog
7	Horse

Class Label	Description
8	Ship
9	Truck

Table 4.1 Labels and Description

4.1.2 Data Preprocessing

Before feeding images into deep learning models, the following pre-processing steps are performed:

1. **Normalization / Scaling**:
    - CNN from scratch: Pixel values scaled to [0,1] by dividing by 255.
    - Pre-trained MobileNetV2: Pixel values processed using preprocess_input() to match the expected input distribution for the pre-trained network.
2. **One-Hot Encoding**:

    Labels are converted into a binary vector of length 10. For example, label 3 (cat) becomes [0,0,0,1,0,0,0,0,0,0].
3. **Data Augmentation**:

    Real-time augmentation applied via ImageDataGenerator to increase dataset diversity:

    - Rotation (±15°)
    - Width and height shifts (±10%)
    - Horizontal flipping

4.1.3 Significance of CIFAR-10

- Widely used benchmark for image classification research**.**
- Suitable for testing and comparing:
    - Custom CNN architectures
    - Transfer learning with pre-trained models
- Provides a controlled environment for evaluating model performance, generalization, and robustness**.**

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENT

5.1 SUMMARY OF FINDINGS

In this study, two approaches for image classification on the CIFAR-10 dataset were investigated: a custom Convolutional Neural Network (CNN) trained from scratch and a pre-trained MobileNetV2 model using transfer learning. Both models were optimized using hyperparameter tuning to identify the best combination of learning rates, dense layer units, convolutional filters, and trainable layers in the case of MobileNetV2. The results revealed that the pre-trained MobileNetV2 consistently outperformed the custom CNN in terms of validation and test accuracy, as well as training efficiency, despite the small image size of CIFAR-10. Data augmentation significantly improved the generalization ability of both models by introducing variability in the training set, reducing overfitting.

The CNN from scratch achieved competitive accuracy but required more epochs and careful tuning of convolutional layers and dense units to reach comparable performance. Hyperparameter tuning demonstrated the importance of selecting optimal learning rates and architecture configurations, as suboptimal choices led to slower convergence and lower test performance. Overall, the study highlighted the efficiency and effectiveness of transfer learning in scenarios with limited dataset size and computational resources, while also showing that well-tuned custom CNNs can achieve reasonable performance.

5.2 CONCLUSION OF FINDINGS

The comparative analysis confirms that transfer learning with pre-trained models like MobileNetV2 provides superior performance and faster convergence for image classification tasks on datasets such as CIFAR-10. The ability to leverage pre-learned features from large-scale datasets (like ImageNet) allows the model to generalize better even with limited data.

However, custom CNNs remain a viable approach, particularly when designing models tailored to specific domains or constraints. Hyperparameter tuning proved to be crucial for both approaches, as the choice of learning rate, dense units, and convolutional filters directly influenced model accuracy and stability.

The study also demonstrated that data augmentation is an effective strategy for improving generalization, reducing overfitting, and enabling the models to handle real-world variations in images.

In summary, pre-trained MobileNetV2 with fine-tuning is recommended for tasks where computational efficiency and high accuracy are desired, while custom CNNs are suitable when a more lightweight or domain-specific architecture is required.

5.3 FUTURE ENHANCEMENT AND SUGGESTIONS

Several directions can be pursued to further improve the models and extend this research:

1. Increase Dataset Complexity: Applying the models to larger or higher-resolution datasets such as CIFAR-100 or ImageNet to evaluate scalability and robustness.
2. Advanced Data Augmentation: Employing techniques like CutMix, MixUp, or Random Erasing to enhance generalization further.
3. Ensemble Learning: Combining multiple CNN architectures or integrating both custom and pre-trained models to boost classification performance.
4. Automated Hyperparameter Optimization: Using more advanced tuning methods such as Bayesian optimization or Hyperband to explore a wider hyperparameter space.

5. Lightweight and Efficient Architectures: Exploring models such as EfficientNet or MobileNetV3 for deployment on resource-constrained devices.
6. Explainability: Applying model interpretability techniques like Grad-CAM to understand which regions of images influence predictions, aiding trust and model refinement.
7. Transfer to Real-World Applications: Extending this approach to domains like medical imaging, surveillance, or autonomous driving to validate performance in practical scenarios.

In conclusion, while the study establishes the effectiveness of pre-trained MobileNetV2 and the viability of custom CNNs, there remains significant potential to improve model robustness, efficiency, and interpretability using the strategies mentioned above.

APPENDIX 1

SAMPLE CODING

```
# ---------------------------------------
# 1. Install required packages (Run once)
# ---------------------------------------
# !pip install tensorflow keras-tuner

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.callbacks import EarlyStopping
import keras_tuner as kt
import numpy as np
import random
import matplotlib.pyplot as plt

# ---------------------------------------
# 2. Fix Random Seeds for Reproducibility
# ---------------------------------------
tf.random.set_seed(42)
np.random.seed(42)
random.seed(42)

# ---------------------------------------
# 3. Load CIFAR-10 Dataset
```

```
# ---------------------------------------
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Convert labels to one-hot
num_classes = 10
y_train = tf.keras.utils.to_categorical(y_train, num_classes)
y_test = tf.keras.utils.to_categorical(y_test, num_classes)

# Preprocess images for MobileNetV2
x_train = preprocess_input(x_train.astype('float32'))
x_test = preprocess_input(x_test.astype('float32'))

print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)

# ---------------------------------------
# 4. Data Augmentation
# ---------------------------------------
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(x_train)

# ---------------------------------------
# 5. Build Pre-trained MobileNetV2 Model
# ---------------------------------------
def build_pretrained_model(hp):
```

```
    base_model = MobileNetV2(
        weights='imagenet',
        include_top=False,
        input_shape=(32, 32, 3)
    )

    # Tune whether to train the base model
    base_model.trainable = hp.Boolean('trainable_base')

    model = models.Sequential()
    model.add(base_model)
    model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(
        units=hp.Int('dense_units', min_value=64, max_value=256, step=64),
        activation='relu'
    ))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(num_classes, activation='softmax'))

    # Tune learning rate
    lr = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# ---------------------------------------
```

```
# 6. Hyperparameter Tuning
# ---------------------------------------
tuner = kt.RandomSearch(
    build_pretrained_model,
    objective='val_accuracy',
    max_trials=2,  # increase for better tuning
    executions_per_trial=1,
    directory='cnn_tuning',
    project_name='cifar10_mobilenetv2'
)

tuner.search(
    datagen.flow(x_train, y_train, batch_size=64),
    epochs=5,
    validation_data=(x_test, y_test)
)

# Display tuning results
print("\nTuning Results Summary:")
tuner.results_summary()

# ---------------------------------------
# 7. Visualize Each Trial's Training History
# ---------------------------------------
trials = tuner.oracle.trials
print("\nTotal Trials:", len(trials))

for trial_id, trial in trials.items():
    print(f"\nTrial: {trial_id}")
    print("Hyperparameters:", trial.hyperparameters.values)
```

```
# Rebuild model with trial hyperparameters
hp = trial.hyperparameters
model = build_pretrained_model(hp)

# Train model to get history
history = model.fit(
    datagen.flow(x_train, y_train, batch_size=64),
    epochs=5,
    validation_data=(x_test, y_test),
    callbacks=[EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)],
    verbose=0
)

# Plot Accuracy & Loss
plt.figure(figsize=(10,4))

# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title(f"Trial {trial_id} Accuracy")
plt.legend()

# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title(f"Trial {trial_id} Loss")
plt.legend()
```

```
    plt.show()

# ---------------------------------------
# 8. Train Final Best Model
# ---------------------------------------
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print("\nBest Hyperparameters Found:")
print(f"Trainable Base: {best_hps.get('trainable_base')}")
print(f"Dense units: {best_hps.get('dense_units')}")
print(f"Learning rate: {best_hps.get('learning_rate')}")

best_model = tuner.get_best_models(num_models=1)[0]

history = best_model.fit(
    datagen.flow(x_train, y_train, batch_size=64),
    epochs=5,
    validation_data=(x_test, y_test)
)

# ---------------------------------------
# 9. Evaluate Final Model
# ---------------------------------------
test_loss, test_acc = best_model.evaluate(x_test, y_test, verbose=2)
print("Final Test Accuracy:", test_acc)

# ---------------------------------------
# 10. Plot Final Accuracy & Loss
# ---------------------------------------
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Final Best Model Accuracy")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Final Best Model Loss")
plt.legend()

plt.show()
```

APPENDIX 2

SCREENSHORT

```
Trial 2 Complete [00h 04m 08s]
val_accuracy: 0.6186000108718872

Best val_accuracy So Far: 0.6186000108718872
Total elapsed time: 00h 08m 42s

Tuning Results Summary:
Results summary
Results in cnn_tuning/cifar10_mobilenetv2
Showing 10 best trials
Objective(name="val_accuracy", direction="max")

Trial 1 summary
Hyperparameters:
trainable_base: True
dense_units: 128
learning_rate: 0.001
Score: 0.6186000108718872

Trial 0 summary
Hyperparameters:
trainable_base: True
dense_units: 256
learning_rate: 0.01
Score: 0.10289999842643738

Total Trials: 2
```

Fig A.1 TRAILS AND SUMMARY

```
Trial: 0
Hyperparameters: {'trainable_base': True, 'dense_units': 256, 'learning_rate': 0.01}
```

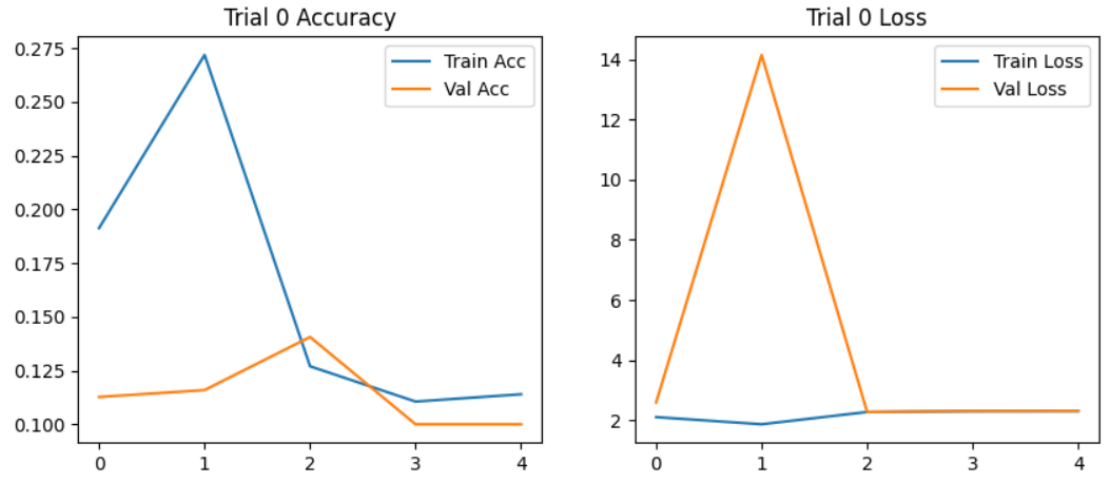

Fig A.2 TRAIL ZERO ACCURACY AND LOSS

```
Trial: 1
Hyperparameters: {'trainable_base': True, 'dense_units': 128, 'learning_rate': 0.001}
```

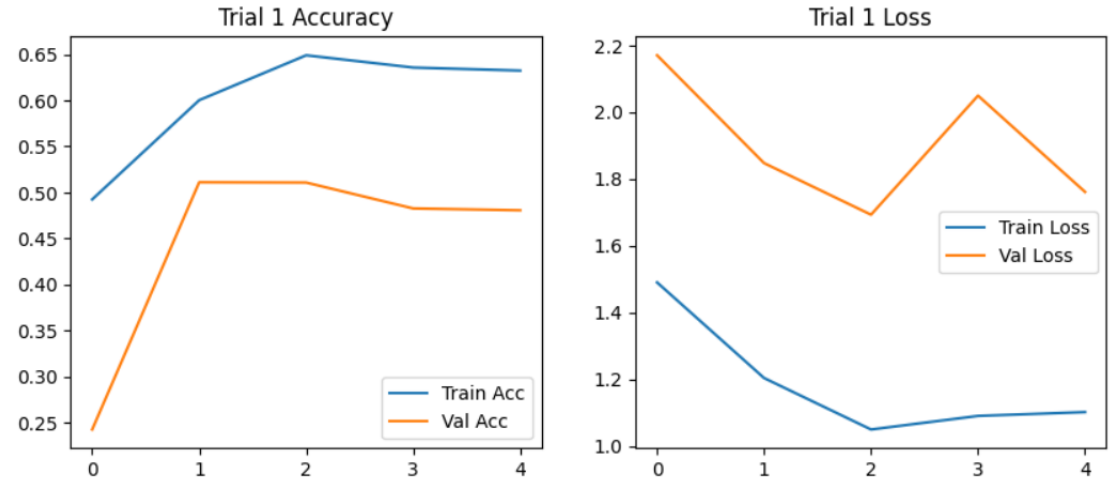

Fig A.3 TRAIL ONE ACCURACY AND LOSS

```
Best Hyperparameters Found:
Trainable Base: True
Dense units: 128
Learning rate: 0.001
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables
  saveable.load_own_variables(weights_store.get(inner_path))
782/782 ━━━━━━━━━━━━━━━━━━━━ 101s 80ms/step - accuracy: 0.6383 - loss: 1.1009 - val_accuracy: 0.3946 - val_loss: 2.5132
Epoch 2/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 36s 46ms/step - accuracy: 0.6499 - loss: 1.0594 - val_accuracy: 0.3887 - val_loss: 3.6109
Epoch 3/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 48ms/step - accuracy: 0.6059 - loss: 1.1887 - val_accuracy: 0.4990 - val_loss: 2.4325
Epoch 4/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 47ms/step - accuracy: 0.6269 - loss: 1.0879 - val_accuracy: 0.5125 - val_loss: 2.6719
Epoch 5/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 47ms/step - accuracy: 0.6791 - loss: 0.9583 - val_accuracy: 0.6665 - val_loss: 1.3508
313/313 - 2s - 5ms/step - accuracy: 0.6665 - loss: 1.3508
Final Test Accuracy: 0.6664999723434448
```

Fig A.4 BEST HYPERPARAMETERS

```
Final Test Accuracy: 0.6664999723434448
```

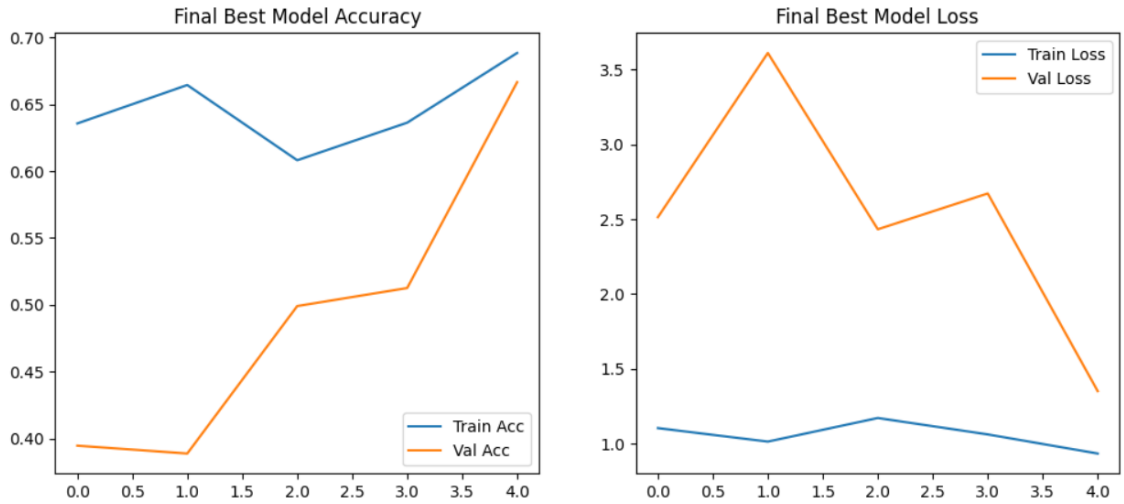

Fig A.5 FINAL BEST MODEL ACCURACY AND LOSS

REFERENCES

[1] Wojciuk et al. (2024) — Improving classification accuracy of fine-tuned CNN models: Impact of hyperparameter optimization.

[2] Krizhevsky (2009) — *CIFAR datasets / Tiny images* (CIFAR-10/100).

[3] Snoek et al. (2015) — Scalable Bayesian optimization for hyperparameter tuning.

[4] Li et al. (2018) / Jamieson et al. — ASHA / Hyperband family (successive halving).

[5] Falkner, Klein & Hutter (2018) — BOHB: combining Bayesian Optimization and Hyperband.

[6] Goodfellow, Bengio & Courville — Deep Learning (2016).

[7] He et al. (2016) — ResNet: Deep residual learning.

[8] Sandler et al. (2018) / MobileNetV2 paper.

[9] Tan & Le (2019) — EfficientNet: Model scaling that matters.

[10] Hutter, Hoos & Leyton-Brown (2014) — fANOVA: assessing hyperparameter importance.

[11] Bergstra & Bengio (2012) — Random Search for Hyper-Parameter Optimization.

[12] Bergstra et al. / Hyperopt (2013) — Practical Bayesian optimization implementations.

[13] Keras Tuner / O'Reilly / TensorFlow docs (practical guides).

[14] Zela, Klein, Falkner & Hutter (2018) — Joint architecture and hyperparameter search.

[15] Garbin et al. / Dropout vs BatchNorm empirical study (2020).

[16] Kingma & Ba (2015) — Adam optimizer.

[17] Srivastava et al. (2014) — Dropout: a simple way to prevent neural networks from overfitting.

[18] Shorten & Khoshgoftaar (2019) — A survey on Image Data Augmentation for Deep Learning.

[19] Perez & Wang (2017) — The Effectiveness of Data Augmentation in Image Classification using Deep Learning.

[20] Zhang et al. (2017) — mixup: Beyond Empirical Risk Minimization.

[21] Howard et al. (2017) — MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

[22] Snoek, Larochelle & Adams (2012) — Practical Bayesian optimization tutorial.

[23] Li et al. (2020) — On the importance of dataset composition for hyperparameter tuning (subset strategies).

[24] Krizhevsky, Sutskever & Hinton (2012) — ImageNet classification with deep convolutional neural networks (AlexNet).

[25] Loshchilov & Hutter (2016) — SGDR: Stochastic Gradient Descent with Warm Restarts (learning-rate schedules).

[26] Molchanov et al. / Han et al. — Model compression & pruning studies.

[27] Ottoni et al. (2023) — Hyperparameter tuning of CNNs for domain-specific image classification.

[28] LeCun, Bengio & Hinton (2015) — Deep learning (Nature).

[29] Li et al. (2018) — Parallelizing Hyperband for large-scale tuning

[30] Recent surveys on AutoML / Neural Architecture Search (NAS) (2018–2022).

SUSTAINABLE DEVELOPMENT GOALS (SDGS)

Goal 9: Industry, Innovation, and Infrastructure

The project primarily aligns with **SDG 9**, which emphasizes building resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering innovation. By developing and optimizing deep learning models for image classification, the project contributes to technological innovation in the following ways:

1. **Promotion of AI and Machine Learning**: Demonstrates practical applications of artificial intelligence (AI) and deep learning, which are key drivers of innovation in multiple industries, including healthcare, agriculture, security, and transportation.
2. **Enhancing Infrastructure for Intelligent Systems**: The methodology of leveraging pre-trained models (transfer learning) and efficient CNN architectures contributes to developing scalable and computationally efficient AI systems, which can be implemented in real-world applications.
3. **Fostering Research and Innovation**: The project explores custom model design**,** hyperparameter tuning and transfer learning**,** promoting experimentation and innovation in AI-driven problem-solving.

Overall, this project **meets SDG 9** by advancing AI-based technological innovation and infrastructure, while also supporting knowledge development (SDG 4) and enabling future applications in sustainable urban solutions (SDG 11). The techniques explored—CNNs, transfer learning, and hyperparameter tuning—provide a foundation for creating innovative solutions that can be deployed across industries and communities to improve efficiency, safety and accessibility.